

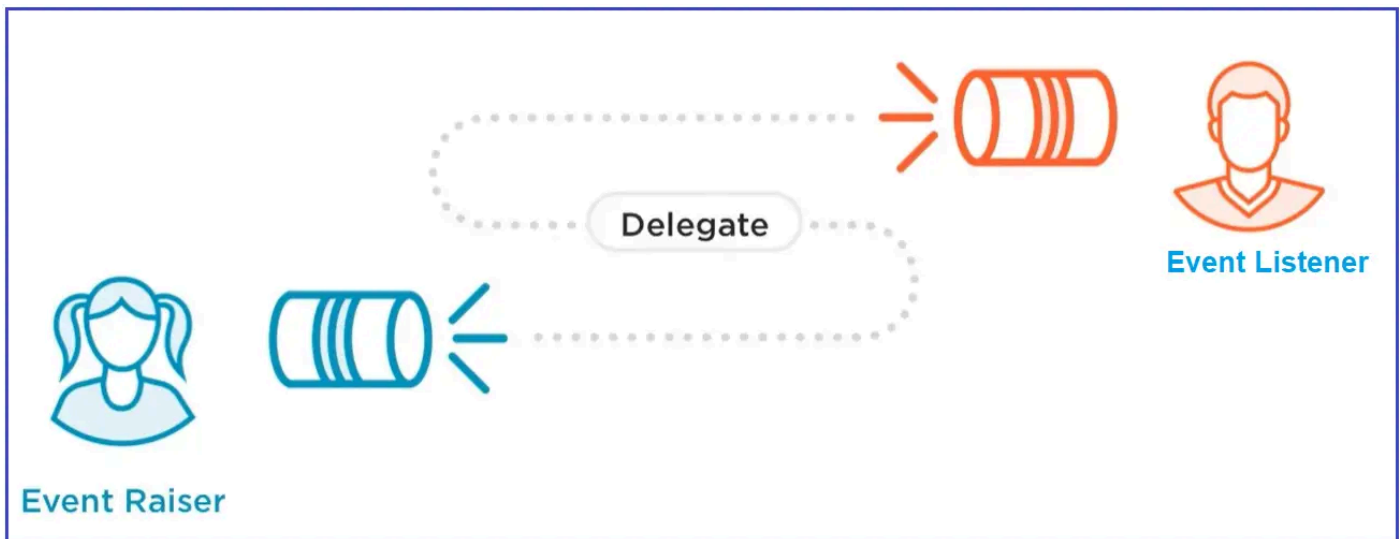
Events and Delegates in C# with Examples

In this article, I am going to discuss **Events and Delegates in C#** with Examples. Please read our previous article where we discussed [Lambda Expressions in C#](#) with Examples. This article is going to be a lengthy one, and please keep patience and read the full article, and I hope at the end of this article, you will understand what exactly events and how to use events, and how events are working in C#.

Events in C#

According to [MSDN](#), Events enable a class or object to notify other classes or objects when something of action occurs. The class that sends (or raises) the event is called the publisher and the classes that receive (or handle) the event are called subscribers. There can be multiple subscribers to a single event. Typically, a publisher raises an event when some action occurred. The subscribers, who are interested in getting a notification when an action occurred, should register with an event and handle it.

We have discussed Delegates and how to create and invoke a delegate in C#. Now, we are going to discuss how to associate a delegate with an event. So that, when an event is raised, we can route event data to a listener. And believe me, it is very easy and at the end of this article, you will understand how to create events and how to associate an event with a delegate, how to raise an event, and how to handle an event in C# with examples.



Defining an Event in C#:

In C#, it's very simple to define an event. What we need to do is, we need to use the event keyword. So, the event can be defined inside a class using the event keyword and the syntax is given below. First, we need to define one delegate, and using that delegate only we need to define an event as shown in the below image.

```
//Step1: Define one delegate
public delegate void WorkPerformedHandler(int hours, WorkType workType);

//Step2: Define one event using the delegate
public event WorkPerformedHandler WorkPerformed;
```

Events are created using event keyword

Delegate

Event Name

As you can see, events are created using the delegate. Here, we created the `WorkPerformed` event using the `WorkPerformedHandler` delegate. This is important because an event on its own does not really do anything. You must have a pipeline or delegate which routes the data from Point A to Point B, that is from Event Raiser to Event Handler or Event Listener and that is going to be our delegate. So, events are really wrappers around the delegate. This is the entire process of defining an event.

Now, you might have one question on your mind, if delegates can use their own, then why events? Although you can use delegates on their own as shown in our previous articles, then why we are using events because they are easy and they are an important part of the dot net framework, and pretty much the standard way to provide notification that is the best way to implement publisher and subscriber pattern.

So, in this case, the listeners can go and attach to the `WorkPerformed` event, but behind the scenes, they are adding themselves to the Invocation List through the `WorkPerformedHandler` delegate that you can see in the event definition.

As per the .NET Framework naming convention, we typically need to add the word `Handler` with our delegate like in our case it is `WorkPerformedHandler`. For naming the event, whatever name you want you can give. This is the simple way how we can define events in C#.

Now, if you want more control over how the listener is added to the Invocation List or removed from the Invocation List, then we need to use add and remove accessors as shown in the below code. But you will not find the below code in the majority of your application or in the sample code.

```

//Step1: Define one delegate
public delegate void WorkPerformedHandler(int hours, WorkType workType);

//Step3: Define one Private field of type delegate
private WorkPerformedHandler _WorkPerformedHandler;

//Step3: Define one event using the delegate
public event WorkPerformedHandler WorkPerformed
{
    [MethodImpl(MethodImplOptions.Synchronized)]
    add
    {
        _WorkPerformedHandler =
            (WorkPerformedHandler)Delegate.Combine(_WorkPerformedHandler, value);
    }

    [MethodImpl(MethodImplOptions.Synchronized)]
    remove
    {
        _WorkPerformedHandler =
            (WorkPerformedHandler)Delegate.Remove(_WorkPerformedHandler, value);
    }
}

```

Here, we have a private field called `_WorkPerformedHandler` and this is nothing but our delegate, using this delegate we are going to add and remove listeners (means methods) from the Invocation List which is going to be called when the event is raised. And within the Event, we have the add accessor and remove accessor. The add accessor will add the listener (technically method) into the Invocation List and the remove accessor will remove the event listener from the Invocation List.

Now, if you notice, we have used the `Synchronized` keyword and this is because we want to make sure that if multiples try to do this at the same time, we synchronize access to this block of code here and that will help us in threading issues.

Now, you can see, within the add accessor, we are simply going to take the value that is trying to be added, which is going to be the target method name and that method should be invoked when the Invocation List is invoked. Here, **Delegate.Combine** will take our delegate that already we have and adds in the value here. This is exactly the same as `+=` that what we have done earlier with multicast delegate. Here, the variable value will hold the target method name that we want to add to the Invocation List.

Now, coming to the remove accessor, it is going to do the same thing as add accessor, but in this case, instead of adding the listener, it will remove the listener from the Invocation List. That means it is the same as `-=` what we discussed earlier with the multicast delegate.

Note: If you have some condition and based on the condition if you want to add or remove the listener, then you need to use add and remove accessors. Or if you want to have more control over which listeners should be added or removed from the Invocation List, then you need to use add and remove accessor. But I never personally use add and remove accessors in my project.

Example to understand How to Create Custom Event in C#:

Let us have a look at how we can add a Custom Event in a C#. Let us add a class file with the name Worker.cs and copy and paste the following code.

```
using System;
namespace DelegatesDemo
{
    //Step1: Define one delegate
    public delegate void WorkPerformedHandler(int hours, WorkType workType);
    public class Worker
    {
        //Step2: Define one event to notify the work progress using the custom
        public event WorkPerformedHandler WorkPerformed;
        //Step2: Define another event to notify when the work is completed using
        EventHandler delegate
        public event EventHandler WorkCompleted;
        public void DoWork(int hours, WorkType workType)
        {
            //Raising Events
        }
    }
    public enum WorkType
    {
        Golf,
        GotoMeetings,
        GenerateReports
    }
}
```

This Worker class is responsible for doing some kind of work. Here we add one method called DoWork which is taking two parameters i.e. int hours, and WorkType workType. Now, who called this DoWork method wants to know how the work is progressing. We cannot return anything from this method until the method execution is completed. But while the method execution is going on, we can raise an event. But, for raising an event, we need a delegate. So, here, first, we are defining a delegate with the name WorkPerformedHandler which is taking two parameters. The following line of code does the same.

```
public delegate void WorkPerformedHandler(int hours, WorkType workType);
```

You can define the delegate anywhere because behind the scene when we compile the application, it will become a class. And once we defined the delegate, then we define an event by using the WorkPerformedHandler delegate and we give the event name as WorkPerformed, and this event is going to be raised while work is progressing. The following line of code does the same.

```
public event WorkPerformedHandler WorkPerformed;
```

Now, we also want to raise an event when the work is completely done. For this, we are creating another event with the name WorkCompleted and this time we are using the built-in delegate EventHandler. If you go to the definition of EventHandler, then you will see that it is a delegate. The following line of code does the same.

```
public event EventHandler WorkCompleted;
```

This is how we define custom events in C#. Now, let us proceed and understand how to Raise Events in C#.

Raising Events in C#:

Once we defined the Events, then we need a way to Raise the Events. And this is important otherwise, the listener would have no way to get the event data. Now, we are going to discuss the different processes or techniques of Raising Events in C#.

Events are Raised simply by calling the events like a method. So, you just need to use the event name and you need to pass the parameter values. Why? Because behind the event, we have delegates only. So, the way we invoke a delegate, in the same way, we need to invoke an event or you can raise an event in C#.

But before raising an event and invoking an event, it's important to check if there is anything in the Invocation List. Because if the delegate behind the scene is null, and if you try to raise the event, then you will get an exception. Why because, if you try to call one method on Null Object, then you know we get a Null Reference Exception in C#. The same thing is also applicable here.

The following piece of code shows how to raise an event in C#. In the below, using the if block we are checking whether the WorkPerformed event is null or not. This will check if anyone is attached to this event who wants to listen. Simply speaking, we are checking if is there any method referred by the delegate. If no listeners are there, then we are not raising the event. This if block statement can be simplified by using the null (?) operator and Invoke method.

```
public void DoWork(int hours, WorkType workType)
{
    //Raising Events
    if (WorkPerformed != null)
    {
        WorkPerformed(8, WorkType.GenerateReports);
    }

    //OR
    WorkPerformed?.Invoke(8, WorkType.GenerateReports);
}
```

So, if the WorkPerformed is not null, then we are raising the event. Raising the event means calling the event just like a method and behind the scene, it just invokes the delegate. And while raising the events, we need to pass the data to the listeners.

Another Way to Raise Events in C#:

Another option is to access the event's delegate and invoke it directly as shown in the below code. As you can see here, we are casting the event to delegate. This is because, if you go to the definition of the event, what is the data type of the event, it is the delegate i.e. WorkPerformedHandler. So, the event can be cast as a delegate. And then we need to check the delegate instance is not null, and if it is not null, then we need to

invoke the delegate just like a method by passing the data to the event listeners. And we can simplify this casting using pattern matching which is also shown here.

```
public void DoWork(int hours, WorkType workType)
{
    WorkPerformedHandler del1 = WorkPerformed as WorkPerformedHandler;
    if (del1 != null)
    {
        del1.Invoke(8, WorkType.GenerateReports);
    }

    //OR Simplify the above using Pattern Matching
    if (WorkPerformed is WorkPerformedHandler del2)
    {
        del2.Invoke(8, WorkType.GenerateReports);
    }
}
```

So, these are the different techniques that you can use to Raise an Event in C#. But, in general, we should not write the Raise event code inside the DoWork Method. What we need to do is, we need to create a new method and within that method, we need to write the code for Raising the Event and we just need to call that method from the DoWork method something as shown in the below code.

```
public void DoWork(int hours, WorkType workType)
{
    //Do Work here and notify the consumer that work has been performed
    OnWorkPerformed(hours, workType);
}

protected virtual void OnWorkPerformed(int hours, WorkType workType)
{
    //Raising Events only if Listeners are attached
    WorkPerformed?.Invoke(8, WorkType.GenerateReports);
}
```

Here, we have created the OnWorkPerformed method which is called by the DoWork method. Now, we have created this OnWorkPerformed method as protected and virtual and the reason is if any subclass of the Worker class wants then can override this method. Now, any listener who has been attached to this event will get notified when we raise this event. Simply speaking the method lists which are added to the Invocation List will execute and all these methods will get the data that are passing while invoking the event. As of now, we have discussed delegates, how to create events and how to raise events. The complete code as of now is given below.

```
using System;
using System.Threading;
namespace DelegatesDemo
```

```

{
    //Step1: Define one delegate
    public delegate void WorkPerformedHandler(int hours, WorkType workType);
    public class Worker
    {
        //Step2: Define one event to notify the work progress using the custom
        public event WorkPerformedHandler WorkPerformed;
        //Step2: Define another event to notify when the work is completed using
        EventHandler delegate
        public event EventHandler WorkCompleted;
        public void DoWork(int hours, WorkType workType)
        {
            //Do Work here and notify the consumer that work has been performed
            for (int i = 0; i < hours; i++)
            {
                OnWorkPerformed(i+1, workType);
                Thread.Sleep(3000);
            }
            //Notify the consumer that work has been completed
            OnWorkCompleted();
        }
        protected virtual void OnWorkPerformed(int hours, WorkType workType)
        {
            //Raising Events only if Listeners are attached
            //Approach1
            //if(WorkPerformed != null)
            //{
            //    WorkPerformed(8, WorkType.GenerateReports);
            //}
            //Approach2
            //WorkPerformed?.Invoke(8, WorkType.GenerateReports);
            //Approach3
            //WorkPerformedHandler del1 = WorkPerformed as WorkPerformedHandler;
            //if(del1 != null)
            //{
            //    del1(8, WorkType.GenerateReports);
            //}
            //Approach4
            if (WorkPerformed is WorkPerformedHandler del2)
            {
                del2(8, WorkType.GenerateReports);
            }
        }
        protected virtual void OnWorkCompleted()
        {
            //Raising the Event
            //Approach1
            //EventHandler delegate takes two parameters i.e. object sender, EventArgs
            //Sender is the current class i.e. this keyword and we do not need to pass
            any data
            //So, we need to pass Empty EventArgs

```

```

        //WorkCompleted?.Invoke(this, EventArgs.Empty);
        //Approach2
        if (WorkCompleted is EventHandler del)
        {
            del(this, EventArgs.Empty);
        }
        //Note: You can also use other two Approached
    }
}
public enum WorkType
{
    Golf,
    GotoMeetings,
    GenerateReports
}
}

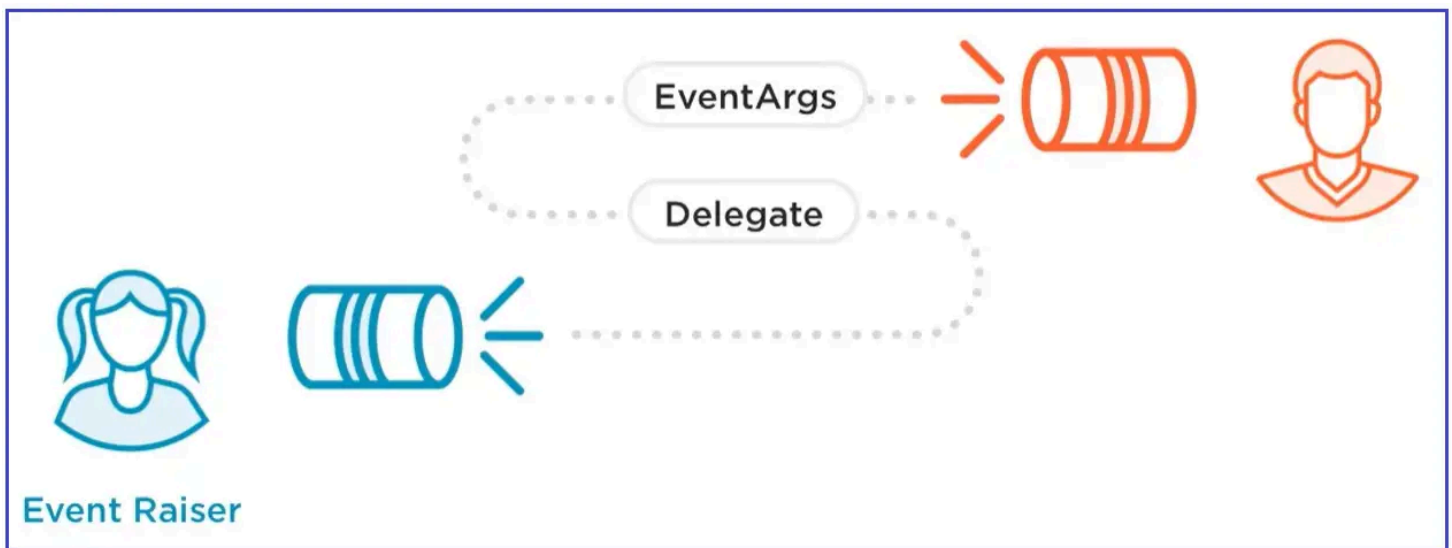
```

Let us discuss each of the methods used in the above code:

1. **DoWork:** This method takes int hours, and WorkType workType as parameters, and inside this method, we are running a loop based on the int hours. And each time we run the loop we call the OnWorkPerformed method and then make the loop sleep for 3 seconds and once the loop is completed, we call the OnWorkCompleted.
2. **OnWorkPerformed:** In this method, we are raising the event and notifying the consumer that the work is going on.
3. **OnWorkCompleted:** In this method, we are raising the event and notifying the consumer that the work is completed. This event is created using the built-in delegate EventHandler which is taking two parameters i.e. object sender, EventArgs e. Here, Sender is the current class i.e. we can pass this keyword and when the work is completed, we don't need to pass any data and we just need to inform that the work is completed. So, we need to pass the second parameter as empty. So, here we are passing EventArgs.Empty as the second parameter.

Creating Custom EventArgs Class in C#:

When we are working with Events in .NET Framework, we need to pass the event data in a specific way. As of now, we have discussed only the simple way to pass the event data like int and WorkType. But what if we want to pass more than 20 or 30 data to the listeners? Then the parameter list in the delegate and in the event handler (event listener) is going to be messy. And, when we raise the event, passing 20 and 30 values are also messy. So, in .NET Framework, we have a standard way to raise the event and pass the event data i.e. by using two parameters i.e. object sender and EventArgs e.



So, what we are going to do is, how we can create our custom EventArgs class and how we can use that custom EventArgs class in our delegate and events to pass data. The EventArgs class is used in the signature of many delegates and event handlers. For example, in Windows Form or Web Form Application, when we double-click on the button element, the button click event is added with the following signature. You can see that it is taking EventArgs as a parameter.

```
public void button_Click(object sender, EventArgs e)
{
    // Handle button click
}
```

Now, if we need to pass our custom data, then we need to extend the EventArgs class as per our business requirements. So, what we need to do is, we need to create a custom EventArgs class by inheriting from the EventArgs class with our required data members as shown in the below image.

```
public class WorkPerformedEventArgs : EventArgs
{
    public int Hours { get; set; }
    public WorkType WorkType { get; set; }
}
```

As you can see, the class is inherited from EventArgs class which belongs to the System namespace. And here we have defined two properties i.e. Hours and WorkType. But, as per your business requirement, you can define any number of properties. Now, add a class file with the name WorkPerformedEventArgs.cs and then copy and paste the following code into it.

```
using System;
namespace DelegatesDemo
{
    public class WorkPerformedEventArgs : EventArgs
```

```

{
    public int Hours { get; set; }
    public WorkType WorkType { get; set; }
}

```

To use the custom EventArgs class, the delegate signature must reference the class in its signature. So, the first parameter is the sender of the event and the second parameter is our custom EventArgs and with this change, the delegate signature must and should look as follows:

```
public delegate void WorkPerformedHandler(object sender, WorkPerformedEventArgs e);
```

So, now when we raise the event, we need to pass two parameters, and who is going to handle this event, he is also having these two parameters.

The .NET Framework includes a generic EventHandler<T> class, that can be used instead of a custom delegate where T represents the EventArgs. So, instead of using our own delegate WorkPerformedHandler while creating the event, we can directly use the generic EventHandler<T> delegate as shown in the below image.

Custom Delegate not Required

```
public delegate void WorkPerformedHandler(object sender, WorkPerformedEventArgs e);
```

Use Generic EventHandler<T> Delegate

```
public EventHandler<WorkPerformedEventArgs> WorkPerformed;
```

Here, T represents the EventArgs

With these changes, while raising the events, we need to pass two parameters as shown in the below code. The first parameter is the sender, in this case, this keyword, and the second parameter is our custom EventArgs.

```

if (WorkPerformed != null)
{
    WorkPerformedEventArgs e = new WorkPerformedEventArgs()
    {
        Hours = hours,
        WorkType = workType
    };
    WorkPerformed(this, e);
}

```

Now, the Worker class code should look as follows:

```

using System;
using System.Threading;
namespace DelegatesDemo
{
    //Step1: Define one delegate
    //Custom Delegate
    //public delegate void WorkPerformedHandler(object sender, WorkPerformedEventArgs e);
    public class Worker
    {
        //Step2: Define one event to notify the work progress using the custom delegate
        public event EventHandler<WorkPerformedEventArgs> WorkPerformed;
        //Step2: Define another event to notify when the work is completed using the
        EventHandler delegate
        public event EventHandler WorkCompleted;
        public void DoWork(int hours, WorkType workType)
        {
            //Do Work here and notify the consumer that work has been performed
            for (int i = 0; i < hours; i++)
            {
                OnWorkPerformed(i+1, workType);
                Thread.Sleep(3000);
            }
            //Notify the consumer that work has been completed
            OnWorkCompleted();
        }
        protected virtual void OnWorkPerformed(int hours, WorkType workType)
        {
            //Raising Events only if Listeners are attached
            //Approach1
            if (WorkPerformed != null)
            {
                WorkPerformedEventArgs e = new WorkPerformedEventArgs()
                {
                    Hours = hours,
                    WorkType = workType
                };
                WorkPerformed(this, e);
            }
            //Approach2
            //EventHandler<WorkPerformedEventArgs> del1 = WorkPerformed as
            //EventHandler<WorkPerformedEventArgs>;
            //if (del1 != null)
            //{
            //    WorkPerformedEventArgs e = new WorkPerformedEventArgs()
            //    {
            //        Hours = hours,
            //        WorkType = workType
            //    };
            //    del1(this, e);
            //}
            //Approach3

```

```

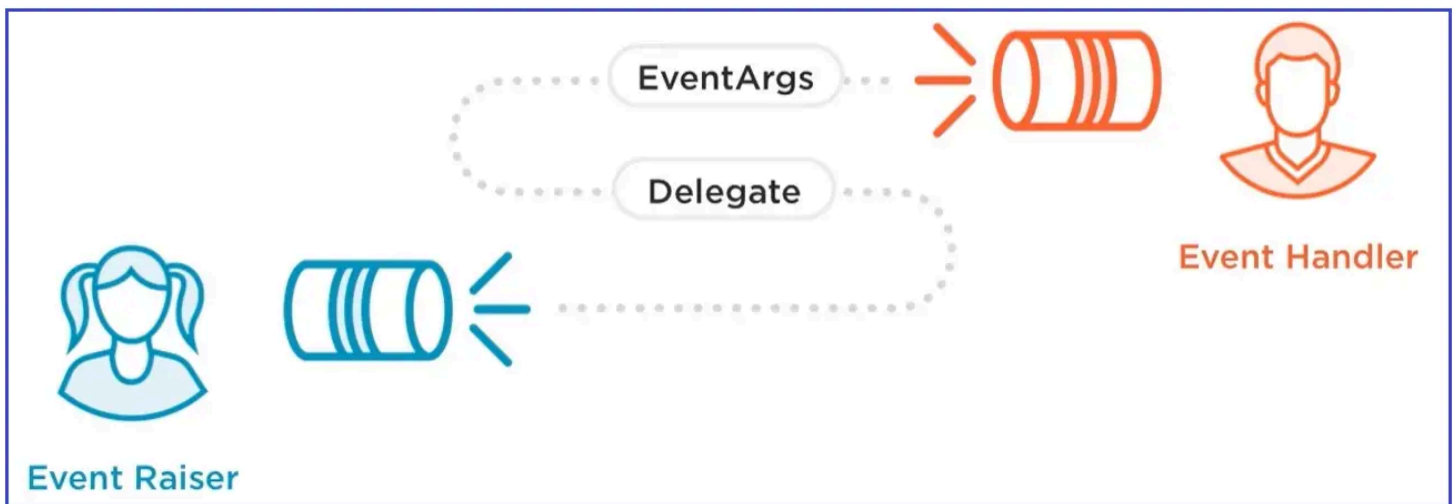
        //if (WorkPerformed is EventHandler<WorkPerformedEventArgs> del2)
        //{
        //    WorkPerformedEventArgs e = new WorkPerformedEventArgs()
        //    {
        //        Hours = hours,
        //        WorkType = workType
        //    };
        //    del2(this, e);
        //}
    }
    protected virtual void OnWorkCompleted()
    {
        //Raising the Event
        //Approach1
        //EventHandler delegate takes two parameters i.e. object sender, EventArgs
        //Sender is the current class i.e. this keyword and we do not need
any data
        //So, we need to pass Empty EventArgs
        //WorkCompleted?.Invoke(this, EventArgs.Empty);
        //Approach2
        if (WorkCompleted is EventHandler del)
        {
            del(this, EventArgs.Empty);
        }
        //Note: You can also use other two Approaches
    }
}
public enum WorkType
{
    Golf,
    GotoMeetings,
    GenerateReports
}
}

```

As of now, we have seen, how to create events, how to create custom events args, and how to raise events. Now, let us proceed and understand the most important things i.e. how to handle the events in C#.

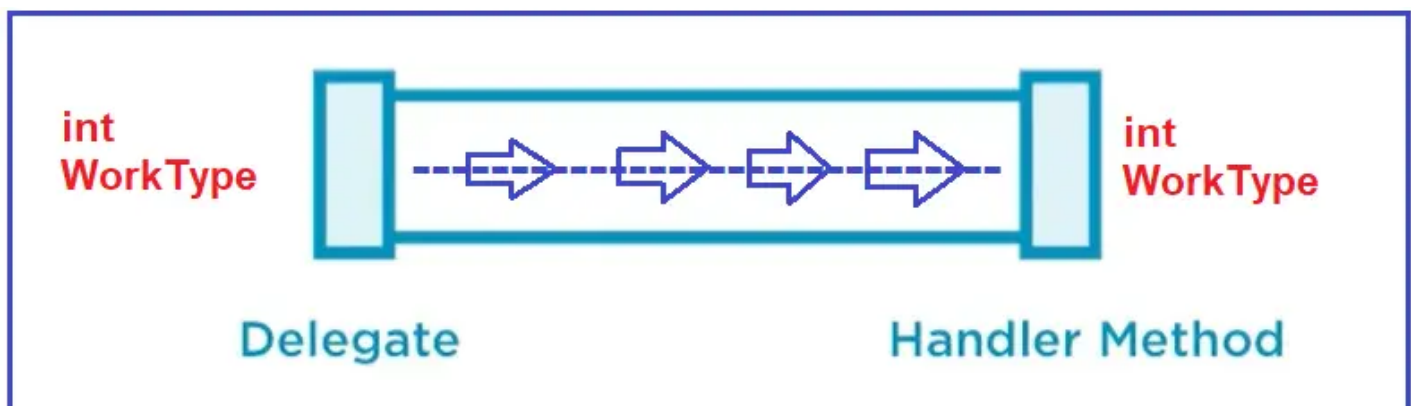
Handling Events in C#:

Now, we are going to discuss how to handle the event in C# which is raised by the Event Raiser.



Instantiating Delegates and Handling Events in C#:

Now, we are going to discuss the process of wiring up an event handler with the event. We have already discussed that the delegate method signature and event handler method signature should and must be the same otherwise the event handler method will not get the data from the delegate or pipeline. If the handler method wants to receive the data from the pipeline, then the handler must have the same number, type, and order of parameters as the delegate. For a better understanding, please have a look at the below image.



In .NET Framework, we can use the `+=` operator to attach an event with an event handler. Here, is an example.

```

static void Main(string[] args)
{
    Worker worker = new Worker();
    worker.WorkPerformed +=  Attached to Event through Generic Delegate
        new EventHandler<WorkPerformedEventArgs>(Worker_Performed);
}

static void Worker_Performed(object sender, WorkPerformedEventArgs e)
{
    Console.WriteLine($"{e.Hours} completed..");
}

```

Note: Signature of EventHandler Delegate and Worker_Performed Method must be same

Now, let us proceed and attached the event handler for the WorkPerformed and WorkCompleted events using the delegate. Please modify the Program class as shown below. Here, we have created two event handler methods. And then attaching the Worker_WorkPerformed event handler method with the WorkPerformed event and attaching the Worker_WorkCompleted method with the WorkCompleted event. For attaching the Worker_WorkPerformed event handler we are using the generic event handler delegate and for the Worker_WorkCompleted event handler method using the built-in Event Handler delegate. And finally, we invoke the DoWork method to start processing.

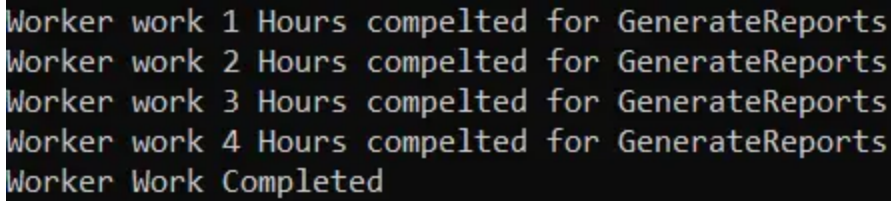
```

using System;
namespace DelegatesDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Worker worker = new Worker();
            //Attaching Worker_WorkPerformed with WorkPerformed Event
            worker.WorkPerformed +=
                new EventHandler<WorkPerformedEventArgs>(Worker_WorkPerformed);
            //Attaching Worker_WorkCompleted with WorkCompleted Event
            worker.WorkCompleted +=
                new EventHandler(Worker_WorkCompleted);
            worker.DoWork(4, WorkType.GenerateReports);
            Console.ReadKey();
        }
        //Event Handler Method or Event Listener Method
        static void Worker_WorkPerformed(object sender, WorkPerformedEventArgs e)
        {
            Console.WriteLine($"Worker work {e.Hours} Hours completed for {e.Hours}");
        }
        //Event Handler Method or Event Listener Method
        static void Worker_WorkCompleted(object sender, EventArgs e)
        {
            Console.WriteLine($"Worker Work Completed");
        }
    }
}

```

```
}  
}
```

Now, run the application and you should get the output as expected as shown in the below image.



```
Worker work 1 Hours compelted for GenerateReports  
Worker work 2 Hours compelted for GenerateReports  
Worker work 3 Hours compelted for GenerateReports  
Worker work 4 Hours compelted for GenerateReports  
Worker Work Completed
```

Attaching Event Handler using Anonymous Method in C#:

An anonymous method is a method that does not have any name. Instead of a standalone method, we can also attach an anonymous method to an event in C#. An anonymous method is created using the delegate keyword.

```
using System;  
namespace DelegatesDemo  
{  
    class Program  
    {  
        static void Main(string[] args)  
        {  
            Worker worker = new Worker();  
            //Attaching Worker_WorkPerformed with WorkPerformed Event  
            worker.WorkPerformed += delegate (object sender, WorkPerformedEventArgs e)  
            {  
                Console.WriteLine($"Worker work {e.Hours} Hours compelted for {e.WorkType}");  
            };  
            //Attaching Worker_WorkCompleted with WorkCompleted Event  
            worker.WorkCompleted += delegate (object sender, EventArgs e)  
            {  
                Console.WriteLine($"Worker Work Completed");  
            };  
            worker.DoWork(4, WorkType.GenerateReports);  
            Console.ReadKey();  
        }  
        ///Event Handler Method or Event Listener Method  
        //static void Worker_WorkPerformed(object sender, WorkPerformedEventArgs e)  
        //{  
        //    Console.WriteLine($"Worker work {e.Hours} Hours compelted for {e.WorkType}");  
        //}  
        ///Event Handler Method or Event Listener Method  
        //static void Worker_WorkCompleted(object sender, EventArgs e)  
        //{  
        //    Console.WriteLine($"Worker Work Completed");  
        //}
```

```
    }  
    //}  
}
```

Output:

```
Worker work 1 Hours compelted for GenerateReports  
Worker work 2 Hours compelted for GenerateReports  
Worker work 3 Hours compelted for GenerateReports  
Worker work 4 Hours compelted for GenerateReports  
Worker Work Completed
```

In the next article, I am going to discuss [Multithreading in C#](#) with Examples. In this article, I try to explain **Events and Delegates in C#** with Examples. I hope you enjoy this Events and Delegates in C# with Examples article.